

MDL-005 — Composition Model

External publication draft

MDL-005 — Composition Model

Interaction explains how the world changes. Composition explains how the world is understood.

Purpose

MDL-001 established **why** Mosaic exists.

MDL-002 established **how** decisions are made.

MDL-003 established **how Mosaic understands the user's world**.

MDL-004 established **how that world behaves**.

MDL-005 defines **how that world is organised into meaningful experiences**.

This specification introduces the Composition Model that transforms:

- World
- Focus
- Context
- Information
- Relationships

into coherent experiences that minimise cognitive effort.

Unlike traditional layout systems, Composition is not concerned with pixels.

It is concerned with meaning.

Relationship to Previous Specifications

Vision

↓

Principles

↓

Mental Model

↓

Interaction Model

↓

Composition Model

↓

Design System

↓

Implementation

The Composition Model assumes:

- the World already exists,
- behaviour has already occurred,
- information has already been understood.

Its responsibility is to organise understanding.

Scope

This specification defines:

- Composition
- Hierarchy
- Priority
- Anchors
- Hero
- Density
- Grouping
- Breathing Space
- Composition Solving
- Adaptive Composition

This specification intentionally does **not** define:

- Components
- Tiles
- Materials
- Motion curves
- Typography
- Colours
- Tokens

Those belong to MDS.

Core Question

MDL-005 exists to answer one question.

How should understanding be organised?

Not:

Where should widgets go?

Composition Statement

Users should never need to consciously search for what matters.

Composition should naturally communicate:

- what is important
- why it is important
- what changed
- what should happen next

without explanation.

Primary Composition Pipeline

World

↓

Focus

↓

Context

↓

Information

↓

Relationships

↓

Priority

↓

Composition

↓

Expressions

↓

Presentation

Composition therefore sits between understanding and interface.

Expected Outcome

After reading MDL-005 contributors should understand:

- how hierarchy emerges
- why hero regions exist

- how adaptive layouts work
- how plugins participate
- how compositions evolve
- how future layout engines should reason

without discussing implementation.

Repository Structure

design/

■■■ mdl/

■■■ MDL-005 Composition Model/

README.md

00-document-control.md

01-what-is-a-composition.md

02-hierarchy.md

03-priority.md

04-hero.md

05-anchors.md

06-adaptive-composition.md

07-density.md

08-breathing-space.md

09-composition-solving.md

10-device-independence.md

11-governance.md

12-adrs.md

13-contributor-guidance.md

glossary.md

references.md

Dependencies

Required reading:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model

Downstream specifications:

- MDS-003 Composition Engine
- MDS-005 Motion System
- MDS-007 Tile Framework
- MDS-008 Component Library

Review Status

Status

Draft

Owner

Lead Design Systems Architect

Next File

00-document-control.md

Document Control

Document Information

Property	Value
Document ID	MDL-005
Title	Mosaic Design Language — Composition Model
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Design Systems Architect
Parent Specifications	MDL-001 Vision, MDL-002 Principles, MDL-003 Mental Model, MDL-004 Interaction Model
Repository	/design/mdl/MDL-005 Composition Model/

Purpose

MDL-005 defines how understanding becomes organisation.

The Mental Model defines:

What exists.

The Interaction Model defines:

How it changes.

The Composition Model defines:

How it should be organised so people understand it.

Composition therefore represents the first specification where abstract understanding becomes a structured experience.

This distinction is fundamental to the Mosaic Design Language.

Authority

The Composition Model governs:

- Information Hierarchy
- Visual Priority
- Structural Relationships
- Spatial Organisation
- Adaptive Density

- Composition Behaviour
- Layout Intent

The Composition Model intentionally does **not** govern:

- Materials
- Colours
- Typography
- Motion Curves
- Rendering Technologies
- Components

Those concerns belong to the Mosaic Design System.

Relationship To MDL

The Composition Model intentionally sits between behaviour and presentation.

[mermaid]
flowchart TD

Vision["MDL-001 Vision"]

Principles["MDL-002 Principles"]

Mental["MDL-003 Mental Model"]

Interaction["MDL-004 Interaction Model"]

Composition["MDL-005 Composition Model"]

DesignSystem["MDS Specifications"]

Implementation["Product"]

Vision --> Principles
Principles --> Mental
Mental --> Interaction
Interaction --> Composition
Composition --> DesignSystem
DesignSystem --> Implementation

Everything beneath MDL-005 assumes that Composition has already determined:

- importance
- hierarchy
- grouping
- emphasis

Presentation merely communicates those decisions.

Design Intent

Most interface systems begin with layout.

Rows.

Columns.

Grids.

Cards.

Containers.

Mosaic intentionally does not.

Composition begins with understanding.

The question is never:

"Where should this card go?"

The question is:

"What deserves attention?"

Only once that question has been answered should presentation become relevant.

Behavioural Scope

This specification governs:

- Hierarchy
- Priority
- Hero
- Anchors
- Adaptive Composition
- Density
- Breathing Space
- Composition Solving

Future MDS specifications implement these concepts visually.

MDL defines them conceptually.

Reader Expectations

Before reading this specification contributors should already understand:

- the Vision
- the Design Principles
- the Mental Model
- the Interaction Model

MDL-005 intentionally assumes familiarity with those concepts.

It extends them.

It does not redefine them.

Stability

Composition should remain significantly more stable than presentation.

Expected lifetime.

Artefact	Expected Lifetime
Components	Months
Layout Implementation	Months
Materials	Months
Composition	Years
Mental Model	Decades

This stability allows Mosaic to evolve visually without changing how information is organised.

Success Criteria

MDL-005 succeeds when:

- users instinctively know where to look
- information naturally forms hierarchy
- compositions adapt without becoming confusing
- plugins integrate without disrupting understanding
- different clients express the same composition consistently

Composition should become invisible.

Users should remember understanding.

Not layout.

Review Status

Status

Draft

Dependencies

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model

Supersedes

None.

Next File

01-what-is-a-composition.md

What Is A Composition?

Purpose

Before discussing hierarchy, priority or adaptive layouts, contributors must first understand what Mosaic means by the word **Composition**.

Within Mosaic, Composition is **not** another name for layout.

Nor is it another name for a screen.

Composition is a conceptual model.

It exists independently from interface technology.

Understanding this distinction is essential because almost every future MDS specification depends upon it.

Definition

Within MDL, a **Composition** is defined as:

The intentional organisation of understanding into a coherent experience.

Notice what this definition intentionally avoids.

A Composition is **not**:

- a page
- a screen
- a layout
- a dashboard
- a collection of components

Those are all implementations.

Composition exists before any of them.

Why Composition Exists

Traditional applications organise interface.

Mosaic organises understanding.

This difference is subtle.

It changes everything.

Traditional thinking:

Layout

↓

Place Widgets

↓

Display Data

Mosaic thinking:

Understanding

↓

Determine Importance

↓

Create Composition

↓

Render Interface

The layout becomes the consequence of understanding.

Not its starting point.

A Composition Is A Story

Every Composition should answer four questions immediately.

1. What currently matters?
2. Why does it matter?
3. What can I do next?
4. Where can I naturally continue?

If users cannot answer those questions within a few seconds, the Composition has failed.

Composition Is Temporary

A Composition is never permanent.

It exists only to communicate the current state of the user's World.

As the World changes:

- Focus changes.
- Context changes.
- Relationships evolve.
- Information becomes available.

The Composition should evolve naturally alongside them.

The World remains.

The Composition changes.

Composition Is Intentional

Nothing exists inside a Composition accidentally.

Every element should justify its presence.

Questions contributors should ask include:

- Why is this visible?
- Why now?
- Why here?
- Why not somewhere else?
- What understanding does it improve?

If those questions cannot be answered, the element probably does not belong.

Composition Is Not Layout

This distinction cannot be overstated.

Layout answers:

Where should something appear?

Composition answers:

Why should something appear?

Many different layouts may correctly express the same Composition.

Likewise...

The same layout may communicate many different Compositions depending upon hierarchy and emphasis.

Composition Is Behaviour

Composition should be viewed as behaviour rather than geometry.

Example.

User begins playback.

Behaviour:

Playback becomes primary.

↓

Timeline reduces.

↓

Progress increases.

↓

Related media becomes secondary.

The Composition has changed.

Nothing in this description references:

- rows
- columns
- spacing
- grids

Those belong to presentation.

Composition Is Hierarchical

Every Composition naturally produces hierarchy.

Examples include:

Primary



Supporting



Contextual



Peripheral

Hierarchy should emerge from:

- Focus
- Context
- Relationships

Never from arbitrary layout decisions.

Composition Is Editorial

Composition is fundamentally an editorial activity.

Editors decide:

- what belongs
- what does not
- what deserves emphasis
- what should wait

Composition performs the same role.

Adding more information rarely improves a Composition.

Choosing the correct information almost always does.

One Composition

Every experience should possess one dominant Composition.

Poor.

Hero

Trending

Downloads

News

Statistics

Plugins

Friends

Updates

Everything competes.

Nothing leads.

Preferred.

Current Focus

↓

Continue

↓

Relevant Information

↓

Related Exploration

One clear story.

One clear hierarchy.

Composition Is Independent From Device

Composition belongs to understanding.

Not hardware.

Desktop.

Television.

Tablet.

Mobile.

Future devices.

All should communicate the same Composition.

Only presentation changes.

Composition Is Independent From Components

Components communicate Composition.

They do not define it.

Example.

Hero

is not a component.

It is a role within a Composition.

Future MDS specifications may implement that role using:

- Tiles
- Shelves
- Posters
- Voice
- Projection

The Composition remains unchanged.

Composition Is Independent From Motion

Motion explains Composition.

It does not create it.

If all animation were removed...

Users should still understand:

- hierarchy
- emphasis
- priority

Motion should reinforce understanding.

Never replace it.

Good Examples

Continue Watching

The Composition immediately communicates:

- current progress
- next episode
- current Focus

Supporting information naturally surrounds that journey.

Everything reinforces continuation.

Reading

The Composition communicates:

- current chapter
- progress
- bookmarks
- related works

The user immediately understands where they are.

Discovering

The Composition shifts naturally towards:

- relationships
- exploration
- connected works

Discovery becomes an extension of the current World.

Not a replacement for it.

Anti-patterns

Dashboard Composition

Attempting to expose every available capability simultaneously.

Understanding collapses.

Equal Weight

Everything appears equally important.

Users must invent their own hierarchy.

Layout Driven

Rows and columns determine hierarchy.

Meaning becomes secondary.

Static Composition

Composition never changes despite changing Context.

The platform gradually becomes less relevant.

Conceptual Model

[mermaid]
flowchart TD

```
World
World --> Focus
Focus --> Context
Context --> Information
Information --> Relationships
Relationships --> Composition
Composition --> Expressions
Expressions --> Presentation
```

Composition represents the final conceptual decision before communication begins.

Litmus Test

Contributors should be able to remove every visual style from a Composition and still answer:

- What matters?
- Why?
- What should I do next?

If not...

The Composition depends upon presentation rather than understanding.

Summary

Composition is the organisation of understanding.

It is not interface.

It is not layout.

It is not implementation.

A successful Composition allows users to instinctively understand their current World before consciously interpreting the interface.

Everything else within MDL-005 builds upon this definition.

Review Status

Status

Draft

Next File

02-hierarchy.md

Hierarchy

Purpose

Hierarchy is the primary mechanism through which a Composition communicates understanding.

Without hierarchy, every piece of information competes equally for attention.

Users are forced to construct their own understanding.

With hierarchy, the Composition communicates understanding before conscious thought occurs.

Hierarchy therefore exists to answer one question.

"What should the user notice first?"

Definition

Within MDL, **Hierarchy** is defined as:

The intentional ordering of attention according to the user's current World.

Hierarchy is determined by:

- Focus
- Context
- Relationships
- User Intent

Hierarchy is **not** determined by:

- available screen space
- arbitrary layout
- implementation convenience
- visual decoration

Presentation communicates hierarchy.

It does not create it.

Why Hierarchy Exists

Every Composition contains more information than users can process simultaneously.

Hierarchy reduces cognitive effort by establishing an order of attention.

Instead of asking:

"Where should the user look?"

contributors should ask:

"What deserves attention?"

The answer naturally produces hierarchy.

Hierarchy Is Meaning

Hierarchy is fundamentally semantic.

The following two interfaces may contain identical information.

Interface A

Movie

Progress

Cast

Runtime

Genre

Reviews

Interface B

Movie

Progress

↓

Next Episode

↓

Related Works

↓

Everything Else

The second communicates understanding.

The first communicates inventory.

Hierarchy transforms information into experience.

Hierarchy Emerges

Hierarchy should emerge naturally from the Mental Model.

World

↓

Focus

↓

Context

↓

Information

↓

Relationships

↓

Hierarchy

↓

Composition

Notice that hierarchy does not begin with interface.

It begins with understanding.

The Four Levels

Every Composition should naturally organise information into four conceptual levels.

Primary

Answers:

What matters now?

Examples include:

- Current Focus
- Active Playback
- Continue Reading

Only one Primary concept should normally exist.

Supporting

Answers:

What helps me continue?

Examples include:

- Progress
- Timeline
- Next Episode
- Chapter Position
- Queue

Supporting information exists because it directly strengthens the Primary concept.

Contextual

Answers:

Why does this matter?

Examples include:

- Cast
- Author
- Studio
- Soundtrack
- Reviews
- Similar Works

Context enriches understanding.

It should not compete with continuation.

Peripheral

Answers:

What else exists?

Examples include:

- Recently Added
- Collections
- Administrative Information
- Background Statistics

Peripheral information should remain available without competing for attention.

One Primary Hierarchy

Every Composition should possess exactly one dominant hierarchy.

Poor.

Trending

Continue Watching

Recently Added

Downloads

Recommendations

Collections

Everything competes.

Nothing leads.

Preferred.

Continue Watching

↓

Next Episode

↓

Timeline

↓

Relationships

↓

Collections

The user's attention naturally flows through the Composition.

Hierarchy Is Dynamic

Hierarchy should evolve continuously.

Example.

Before playback.

Episode

↓

Synopsis

↓

Runtime

During playback.

Playback

↓

Progress

↓

Remaining Time

After playback.

Next Episode

↓

Progress

↓

Related Works

The information remains largely identical.

Its hierarchy changes.

Hierarchy Is Relative

Importance is always relative.

Example.

Runtime

Before playback.

Useful.

During playback.

Almost irrelevant.

Likewise.

Cast

While exploring.

Important.

While watching.

Peripheral.

Hierarchy should therefore remain contextual rather than absolute.

Hierarchy Across Domains

Hierarchy should remain conceptually consistent across every entertainment domain.

Television.

Current Episode

↓

Progress

↓

Next Episode

Books.

Current Chapter

↓

Progress

↓

Next Chapter

Music.

Current Track

↓

Playback

↓

Queue

Different information.

Identical hierarchy.

This consistency strengthens the Mental Model.

Hierarchy Is Behaviour

Hierarchy should change before presentation changes.

Example.

Episode Released

↓

Priority Increases

↓

Hierarchy Changes

↓

Presentation Updates

The behavioural decision precedes the interface.

Future Composition Engines should preserve this ordering.

Good Examples

Watching

Primary

Playback

↓

Supporting

Progress

↓

Contextual

Characters

↓

Peripheral

Collections

Reading

Primary

Current Chapter

↓

Supporting

Bookmarks

↓

Contextual

Author

↓

Peripheral

Library

Exploring

Primary

Current Focus

↓

Supporting

Relationships

↓

Contextual

Metadata

↓

Peripheral

History

The behavioural pattern remains identical.

Anti-patterns

Equal Importance

Every item receives identical emphasis.

The user must invent hierarchy.

Decorative Hierarchy

Hierarchy created solely through colour or animation.

Meaning should exist before presentation.

Static Hierarchy

Hierarchy never changes despite changing Context.

The interface gradually becomes irrelevant.

Multiple Primaries

Several competing concepts all attempt to become the centre of attention.

The Composition loses clarity.

Hierarchy Model

[mermaid]
flowchart TD

Focus
Focus --> Context
Context --> Priority
Priority --> Hierarchy
Hierarchy --> Composition
Composition --> Presentation

Hierarchy translates priority into understanding.

Presentation merely communicates it.

Relationship To Priority

Hierarchy and Priority are closely related.

They are not identical.

Priority answers:

How important is this?

Hierarchy answers:

How should that importance be communicated?

Priority is an internal decision.

Hierarchy is an organisational decision.

This distinction becomes important in the next chapter.

Summary

Hierarchy organises understanding.

It allows users to instinctively know:

- what matters
- why it matters
- what deserves attention next

without consciously interpreting the interface.

Hierarchy should emerge from the user's World.

Never from arbitrary interface structure.

Review Status

Status

Draft

Next File

03-priority.md

Priority

Purpose

Hierarchy determines **how understanding is organised**.

Priority determines **why one piece of information deserves more attention than another**.

Priority is one of the core behavioural inputs to the Composition Model.

Without Priority, every Composition becomes arbitrary.

With Priority, hierarchy emerges naturally from the user's World.

Priority therefore answers one question.

"How important is this information right now?"

Definition

Within MDL, **Priority** is defined as:

The relative importance of Information within the user's current World.

Priority is dynamic.

It changes as:

- Focus changes
- Context changes
- Relationships evolve
- Time passes
- User intent changes

Priority is never fixed.

Why Priority Exists

A World contains significantly more Information than can be communicated simultaneously.

Example.

Current Episode

Cast

Reviews

Genres

Runtime

Artwork

Next Episode

Characters

Soundtrack

Studio

Progress

Collections

History

Every item is useful.

Not every item is equally important.

Priority allows Mosaic to determine:

- what deserves immediate attention
- what should remain visible
- what may quietly recede
- what should remain available but peripheral

Priority Exists Before Hierarchy

Priority should always be determined before hierarchy.

World

↓

Focus

↓

Context

↓

Information

↓

Relationships

↓

Priority

↓

Hierarchy

↓

Composition

Hierarchy communicates Priority.

It should never invent it.

Priority Is Contextual

Priority is not an inherent property of Information.

It is a property of the current experience.

Example.

Runtime

Browsing a film.

Medium Priority.

Watching the film.

Low Priority.

Likewise.

Next Episode

Before finishing the current episode.

Low Priority.

After finishing.

Very High Priority.

The Information remains identical.

Priority changes.

Priority Levels

The Composition Model defines four conceptual priority levels.

Critical

Information required immediately.

Examples.

- Active playback
- Reading position
- Current Focus

A Composition should rarely contain more than one or two Critical concepts.

High

Information directly supporting the current activity.

Examples.

- Progress
- Timeline

- Next Episode
- Queue
- Continue Reading

High Priority information should remain easily discoverable.

Medium

Information enriching the current experience.

Examples.

- Cast
- Author
- Reviews
- Related Works
- Studio

Medium Priority should support exploration.

It should not interrupt continuation.

Low

Information valuable only in specific situations.

Examples.

- Technical metadata
- Collections
- Administrative information
- Historical statistics

Low Priority information should remain accessible without competing for attention.

Priority Is Relative

Priority should always be evaluated relative to the current World.

Example.

Watching

↓

Playback

Critical

↓

Reviews

Medium

Later.

Browsing Reviews

↓

Reviews

High

↓

Playback

Low

Nothing changed except the user's intent.

Priority naturally followed.

Priority Is Temporal

Time influences Priority.

Example.

Episode

↓

Releases

Tomorrow

Priority:

Medium.

Episode

↓

Available

Now

Priority:

High.

Episode

↓

Watched

Priority:

Low.

Temporal behaviour therefore continuously influences Priority without changing the underlying Information.

Priority Is Behaviour

Priority is a behavioural decision.

It should never be determined purely by presentation.

Poor.

Large Tile

↓

Must Be Important

Correct.

Important

↓

Receives Larger Expression

Presentation reflects Priority.

It does not determine it.

Priority And Plugins

Extensions should never assign visual priority.

Instead they contribute Information.

The platform evaluates Priority using:

- current Focus
- current Context
- existing Relationships
- temporal relevance
- user intent

This ensures every plugin behaves consistently.

Priority Compression

When space becomes limited:

Priority determines what compresses first.

The preferred order is:

1. Low 2. Medium 3. High 4. Critical

Critical information should disappear only when no alternative exists.

Compression should preserve understanding before preserving completeness.

Priority Expansion

When more space becomes available:

Priority determines what expands first.

Expansion order:

1. Critical 2. High 3. Medium 4. Low

Additional space should reveal deeper understanding.

Not unrelated information.

Good Examples

Watching

Critical

Playback

↓

High

Progress

↓

High

Next Episode

↓

Medium

Characters

↓

Low

Collections

Reading

Critical

Current Chapter

↓

High

Bookmarks

↓

Medium

Author

↓

Low

Reading Statistics

Exploring

Critical

Current Focus

↓

High

Relationships

↓

Medium

Metadata

↓

Low

Administration

Priority adapts naturally to intent.

Anti-patterns

Fixed Priority

Information always appears with identical importance.

The Composition becomes inflexible.

Popularity Priority

Trending information overrides the user's current World.

Platform goals replace user goals.

Visual Priority

Colour, animation or size artificially create importance without conceptual justification.

Meaning should always precede presentation.

Unlimited High Priority

Everything attempts to become important.

Hierarchy collapses.

Understanding decreases.

Priority Model

[mermaid]
flowchart TD

```
World
World --> Focus
Focus --> Context
Context --> Information
Information --> Relationships
Relationships --> Priority
Priority --> Hierarchy
Hierarchy --> Composition
```

Priority is the bridge between understanding and organisation.

Relationship To Hero

The Hero is not manually chosen.

It naturally emerges from Priority.

The highest-priority concept within the current Composition generally becomes the Hero.

This relationship is formalised in the next chapter.

Summary

Priority determines the importance of Information within the user's current World.

It is dynamic.

It is contextual.

It is behavioural.

Hierarchy communicates Priority.

Composition organises it.

Presentation expresses it.

This separation allows Mosaic to remain adaptive while preserving a stable conceptual model.

Review Status

Status

Draft

Next File

04-hero.md

Hero

Purpose

Every Composition should possess a single, unmistakable centre of gravity.

Within the Mosaic Design Language this concept is known as the **Hero**.

Unlike traditional web design, the Hero is **not** a banner.

It is **not** the first section on a page.

It is **not** a marketing device.

Within Mosaic, the Hero is the visual manifestation of the most important concept within the current Composition.

The Hero exists to answer one question.

"What currently deserves the user's attention?"

Definition

Within MDL, a **Hero** is defined as:

The highest-priority Expression within the current Composition.

The Hero is not manually positioned.

It naturally emerges from:

- Focus
- Context
- Priority
- Relationships

The Hero is therefore an outcome.

Not a component.

Why A Hero Exists

Human attention naturally seeks a starting point.

Without a clear starting point users must first determine:

- where to look
- what matters
- what is secondary

This unnecessary interpretation increases cognitive effort.

A Hero removes that uncertainty.

It immediately establishes:

- the current Focus
- the current journey
- the current story

Well-established design systems use hero sections to establish the primary message and create an immediate visual hierarchy, but Mosaic extends the concept beyond page introductions into a dynamic behavioural role.

[oai_citation:0±Sky UI](https://sky-ui.cf.sky.com/sections/hero?utm_source=chatgpt.com)

The Hero Is Not A Banner

Traditional Hero sections introduce websites.

Mosaic Heroes introduce understanding.

Traditional:

Page

↓

Hero Banner

↓

Content

Mosaic:

Composition

↓

Hero

↓

Supporting Information

The Hero belongs to the Composition.

Not the page.

The Hero Is Dynamic

The Hero should evolve naturally.

Example.

Current Context.

Browsing Series

Hero.

Frieren

Beginning playback.

The Hero evolves into:

Current Episode

Finishing playback.

The Hero evolves into:

Next Episode

The Hero should follow the user's journey rather than remaining static.

One Hero

Every Composition should contain one primary Hero.

Poor.

Trending

Continue Watching

Featured Movie

Latest Release

Downloads

Several competing heroes.

Preferred.

Continue Watching

↓

Everything Else

The user's attention naturally begins in the correct place.

Hero Emerges From Priority

The Hero is never manually selected.

Instead:

[text]
Information

↓

Relationships

↓

Priority

↓

Highest Priority

↓

Hero

Changing Priority naturally changes the Hero.

No additional behavioural rules are required.

Hero Does Not Mean Largest

The Hero communicates importance.

Not physical size.

Examples.

Desktop.

Large Artwork

Television.

Immersive Poster

Mobile.

Compact Hero

Voice.

Continue watching Frieren?

Every implementation communicates the same Hero.

Presentation adapts.

Meaning remains constant.

Hero Owns Attention

The Hero establishes the initial attention path.

Everything else should reinforce it.

Supporting information should answer questions generated by the Hero.

Poor.

Hero

↓

Unrelated Downloads

↓

Random Recommendations

Preferred.

Hero

↓

Progress

↓

Timeline



Related Works

The Composition tells one coherent story.

Hero And Artwork

Artwork naturally belongs within the Hero.

Artwork already communicates:

- identity
- emotion
- atmosphere

The interface should avoid competing with this.

Future Material specifications should therefore derive atmosphere from Hero artwork rather than imposing unrelated visual identity.

The Hero should feel like entertainment.

Not software.

Hero And Movement

Movement should preserve Hero identity.

Example.

Episode



Playback

The Hero evolves.

It should not disappear and be replaced.

Users should always understand:

"This is still the same experience."

Movement preserves Hero continuity.

Hero Across Domains

Every Domain should possess an equivalent Hero.

Television.

Current Episode

Books.

Current Chapter

Music.

Currently Playing

Movies.

Current Film

Different media.

Identical behavioural role.

This consistency strengthens the Mental Model.

Hero Lifetime

Heroes are intentionally temporary.

They exist only while they represent the highest-priority concept.

As Priority changes...

The Hero changes.

As Context changes...

The Hero evolves.

As Focus changes...

A new Hero naturally emerges.

This continual evolution reflects the user's World rather than arbitrary interface state.

Good Examples

Continue Watching

Hero.

Current Episode

Supporting.

- Progress
- Timeline
- Next Episode

Everything strengthens continuation.

Reading

Hero.

Current Chapter

Supporting.

- Bookmarks
- Reading Progress
- Remaining Chapters

Again...

One clear centre.

Discovering

Hero.

Current Focus

Supporting.

- Relationships
- Adaptations
- Similar Works

Discovery grows naturally from the Hero.

Anti-patterns

Multiple Heroes

Several competing concepts all demand primary attention.

Hierarchy collapses.

Promotional Hero

Trending content replaces the user's current Focus.

The platform has begun persuading rather than accompanying.

Static Hero

The Hero never changes despite changing Context.

The Composition becomes increasingly disconnected from the user's World.

Decorative Hero

The Hero exists only because it looks visually impressive.

Importance should determine the Hero.

Not aesthetics.

Hero Model

```
[mermaid]
flowchart TD
    World
    World --> Focus
    Focus --> Context
    Context --> Priority
    Priority --> Hero
    Hero --> SupportingInformation["Supporting Information"]
    SupportingInformation["Supporting Information"] --> Composition
```

The Hero emerges naturally from understanding.

It is never manually inserted into the interface.

Summary

The Hero is the visual centre of the current Composition.

It communicates:

- what matters
- why it matters
- where the user's attention should begin

A Hero should never exist because every application has one.

It exists because every human naturally looks for one place to begin.

Within Mosaic, the Hero provides that beginning.

Review Status

Status

Draft

Next File

05-anchors.md

Anchors

Purpose

If the Hero establishes the centre of gravity within a Composition, **Anchors** establish stability.

Without Anchors, adaptive interfaces become disorientating.

Everything moves.

Nothing feels reliable.

With Anchors, the Composition may evolve significantly while users retain a strong sense of orientation.

Anchors are therefore the mechanism through which Mosaic balances adaptability with predictability.

Definition

Within MDL, an **Anchor** is defined as:

A stable conceptual region whose primary responsibility is preserving orientation while the surrounding Composition evolves.

Notice the wording.

An Anchor is **not**:

- a fixed component
- a fixed location
- a fixed piece of interface

It is a stable point of understanding.

Presentation may differ.

Purpose does not.

Why Anchors Exist

Adaptive interfaces naturally introduce movement.

Movement improves understanding only while users continue recognising where they are.

Anchors provide that recognition.

They answer questions such as:

- Where am I?
- Am I still inside the same World?
- Has everything changed?
- What stayed the same?

Without Anchors, users repeatedly reconstruct their mental model.

With Anchors, users simply observe the World evolving.

Stable anchor points are widely recognised as one of the primary mechanisms for preserving users' spatial memory while interfaces adapt. [oai_citation:0±uxmatters.com](https://www.uxmatters.com/mt/archives/2026/05/designing-stable-user-interfaces.php?utm_source=chatgpt.com)

Stability Before Position

Anchors should be understood conceptually rather than geometrically.

Poor definition.

Navigation

Always Left

Better definition.

Navigation

Always Stable

Whether Navigation appears:

- left
- bottom
- television overlay
- voice interface

its behavioural role remains identical.

The Anchor survives.

Presentation changes.

Types Of Anchors

Every Composition should naturally contain several Anchor types.

Navigation Anchor

Purpose:

Maintain orientation within the user's World.

Responsibilities:

- Domains
- Global navigation
- Current location

The Navigation Anchor should move rarely.

Focus Anchor

Purpose:

Continuously communicate the current Focus.

Examples include:

- Current Series
- Current Book
- Current Album

Even while surrounding information evolves, users should immediately recognise what currently matters.

Playback Anchor

Purpose:

Preserve continuity during consumption.

Examples include:

- playback controls
- progress
- current chapter

Playback should remain behaviourally stable while surrounding Composition adapts.

Search Anchor

Purpose:

Provide predictable access to discovery.

Search should never become difficult to locate simply because Context changed.

Identity Anchor

Purpose:

Communicate whose World is currently being experienced.

Future capabilities such as:

- Family Worlds
- Shared Worlds
- Child Worlds

will rely heavily upon Identity Anchors.

Anchors And Adaptation

Anchors intentionally resist change.

Everything else remains adaptive.

Conceptually.

Anchors

↓

Stable

Adaptive Regions

↓

Flexible

The adaptive parts of the Composition evolve around the Anchors.

Not the reverse.

Anchors And Behaviour

Anchors should change only when behaviour genuinely changes.

Example.

Watching.

Playback

↓

Stable

Browsing metadata.

Playback should remain recognisable.

Changing Domains.

Navigation remains stable.

Current Focus changes.

Anchors preserve understanding while adaptive information reorganises.

Anchors Across Devices

Anchors should preserve behavioural identity rather than position.

Desktop.

Navigation

Left

Mobile.

Navigation

Bottom

Television.

Navigation

Overlay

The behavioural role remains identical.

Only presentation changes.

This distinction is fundamental.

Anchors And Composition

Anchors exist outside ordinary hierarchy.

Hierarchy answers:

What deserves attention?

Anchors answer:

What should remain understandable?

An Anchor may therefore possess lower visual priority while remaining behaviourally critical.

Anchors And Hero

Hero and Anchor intentionally solve different problems.

Hero	Anchor
Communicates attention	Preserves orientation
Dynamic	Stable
Emerges from Priority	Exists independently of Priority
Frequently changes	Rarely changes

Both are required.

Without Hero...

Nothing leads.

Without Anchors...

Everything drifts.

Good Examples

Watching

Stable.

- Navigation
- Playback
- Search

Adaptive.

- Timeline
- Related Works
- Cast
- Recommendations

Users remain oriented while information evolves.

Reading

Stable.

- Reading Progress
- Search
- Current Book

Adaptive.

- Author
- Notes
- Related Series
- Characters

Again...

Stability and adaptation coexist.

Domain Change

Changing from:

Anime

↓

Books

The Navigation Anchor remains recognisable.

The adaptive Composition reorganises.

Users understand they changed Domain.

Not product.

Anti-patterns

Everything Moves

Every element repositions during adaptation.

Users lose spatial memory.

No Stable Regions

The interface possesses no reliable reference points.

Every interaction requires reconstruction.

Decorative Anchors

Large visual regions possessing no behavioural responsibility.

Anchors exist to preserve understanding.

Not symmetry.

Anchors Competing With Hero

Stable elements continually compete with current Focus.

Orientation begins overwhelming attention.

Behavioural Model

[mermaid]
flowchart TD

StableAnchors

--> Composition

AdaptiveRegions

--> Composition

Composition

--> UserUnderstanding

Anchors preserve.

Adaptive regions evolve.

Together they produce understandable interaction.

Relationship To Future Specifications

Future specifications should treat Anchors as first-class behavioural concepts.

Examples include:

- MDS Composition Engine
- Adaptive Layout Solver
- Motion System
- Responsive Behaviour
- Accessibility

Every future adaptive system should preserve Anchors before optimising presentation.

Summary

Anchors are the stable reference points within the user's World.

They allow Mosaic to become highly adaptive without becoming unpredictable.

The more adaptive the platform becomes...

The more valuable Anchors become.

They are the quiet constants that allow everything else to evolve naturally.

Review Status

Status

Draft

Next File

06-adaptive-composition.md

Adaptive Composition

Purpose

A Composition should never be considered a fixed arrangement of interface elements.

The user's World is constantly evolving.

Devices differ.

Contexts change.

Relationships emerge.

Space changes.

Plugins contribute new information.

Adaptive Composition exists to ensure that the organisation of understanding evolves naturally without requiring contributors to design every possible state individually.

Unlike responsive layout systems, Adaptive Composition does **not** begin with screen size.

It begins with understanding.

Definition

Within MDL, **Adaptive Composition** is defined as:

The ability of a Composition to continuously reorganise itself while preserving understanding.

Adaptation should always preserve:

- hierarchy
- continuity
- orientation
- intent

Adaptation should never feel random.

Why Adaptive Composition Exists

Traditional interfaces often behave like templates.

Desktop Layout

↓

Tablet Layout

↓

Phone Layout

Every device receives a different layout.

Mosaic intentionally adopts a different approach.

One Composition

↓

Many Presentations

The Composition remains conceptually identical.

Presentation adapts.

This distinction allows the user's World to remain recognisable regardless of device.

Adaptation Begins With Meaning

Adaptive Composition should never ask:

"How much space do we have?"

Instead it asks:

"What currently deserves attention?"

Only after hierarchy has been established should available space influence presentation.

Meaning therefore precedes geometry.

Inputs

Adaptive Composition evaluates the following conceptual inputs.

[text]
World

↓

Focus

↓

Context

↓

Priority

↓

Relationships

↓

Available Space

↓

Composition

Notice that available space appears almost at the end.

The user's World remains the primary input.

Adaptation Triggers

Adaptive Composition should respond naturally to changes in:

User Behaviour

Examples include:

- beginning playback
- completing playback
- changing Focus
- changing Domain
- opening supporting information

Time

Examples include:

- new episode released
- countdown completed
- metadata updated
- scheduled event

Device

Examples include:

- phone
- tablet
- television
- desktop

Environment

Future implementations may also adapt according to:

- orientation
- accessibility preferences
- available display area
- viewing distance

These are implementation inputs.

They should never override the user's World.

Adaptive Rules

Adaptive Composition should follow several behavioural rules.

Rule One

Preserve understanding.

Understanding should never decrease simply because adaptation occurred.

Rule Two

Preserve hierarchy.

Primary concepts should remain primary.

Supporting concepts should remain supporting.

Rule Three

Preserve continuity.

Users should understand why adaptation occurred.

Rule Four

Adapt locally before globally.

Small changes should produce small adaptations.

Rule Five

Reduce before removing.

Compression should always be preferred over disappearance.

Adaptation Strategies

Adaptive Composition possesses several conceptual strategies.

Compression

Information communicates less while remaining present.

Example.

Timeline.

10 Episodes

↓

3 Episodes

↓

1 Episode

The concept survives.

Its expression changes.

Expansion

Additional understanding becomes available.

Example.

Progress.

Percentage

↓

Percentage + Chapter

↓

Percentage + Chapter + Remaining Time

The same concept simply becomes richer.

Reordering

Priority changes.

The Composition reorganises.

No information is lost.

Grouping

Related Information combines into larger conceptual units.

Grouping reduces cognitive effort while preserving understanding.

Progressive Disclosure

Information remains available.

It simply waits until it becomes useful.

This should always be preferred over immediate overload.

Compression Hierarchy

When adaptation becomes necessary, concepts should compress in the following order.

Low Priority

↓

Medium Priority

↓

Supporting Information

↓

High Priority

↓

Critical Information

Critical understanding should remain visible for as long as possible.

Expansion Hierarchy

As additional space becomes available:

Critical

↓

Supporting

↓

Contextual

↓

Peripheral

The user's understanding should deepen naturally.

Not broaden randomly.

Device Independence

Adaptive Composition intentionally separates:

Composition

↓

Presentation

Desktop.

Large Hero

Expanded Timeline

Rich Relationships

Mobile.

Compact Hero

Condensed Timeline

Progressive Relationships

Television.

Immersive Hero

Minimal Navigation

Large Artwork

The Composition remains identical.

Only expression changes.

Plugins

Extensions should not adapt themselves.

Instead they contribute:

- Information
- Relationships

The platform evaluates:

- hierarchy
- priority
- available space
- accessibility

This guarantees one coherent adaptive system across the entire platform.

Good Examples

Phone

Limited space.

Timeline compresses.

Relationships progressively disclose.

Current Focus remains obvious.

Desktop

Additional space.

Timeline expands.

Relationships deepen.

Artwork grows.

The World becomes richer.

Television

Distance increases.

Typography enlarges.

Peripheral information compresses.

Hero becomes dominant.

The same Composition adapts naturally.

Anti-patterns

Responsive Templates

Creating unrelated layouts for every device.

The user's World fragments.

Random Adaptation

Information appears and disappears unpredictably.

Understanding decreases.

Priority Reversal

Less important information becomes dominant simply because additional space exists.

Meaning has been replaced by geometry.

Plugin Layout

Extensions independently adapting interface.

Adaptive behaviour becomes inconsistent.

Adaptive Behaviour Model

[mermaid]
flowchart TD

```
World
World --> Priority
Priority --> AvailableSpace["Available Space"]
AvailableSpace["Available Space"] --> AdaptiveComposition["Adaptive Composition"]
AdaptiveComposition["Adaptive Composition"] --> Expressions
Expressions --> Presentation
```

Adaptation should always preserve the conceptual model.

Presentation changes.

Understanding does not.

Summary

Adaptive Composition allows one conceptual experience to exist across many different devices and situations.

Adaptation should always strengthen understanding.

Never weaken it.

Users should recognise their World immediately regardless of:

- screen size
- device
- orientation
- accessibility preferences

Adaptive Composition therefore represents one of the defining architectural capabilities of Mosaic.

Review Status

Status

Draft

Next File

07-density.md

Density

Purpose

Density describes **how much understanding should be communicated at one time**.

Within traditional user interface design, density is frequently treated as a visual concern.

Examples include:

- spacing
- margins
- compact mode
- comfortable mode

Within the Mosaic Design Language, density is **not** a visual property.

Density is a property of the Composition.

It determines how much understanding should be communicated before presentation begins.

Definition

Within MDL, **Density** is defined as:

The amount of conceptual information intentionally communicated within a Composition.

Density describes understanding.

Not pixels.

Not spacing.

Not screen size.

Why Density Exists

The amount of information appropriate for one situation is rarely appropriate for another.

Examples.

Watching a film.

The user typically needs:

- playback
- progress
- subtitles
- next episode

Everything else should quietly recede.

Exploring a franchise.

The user now expects:

- relationships
- chronology
- cast
- soundtrack
- adaptations
- production

The Composition naturally becomes denser.

The difference is driven by intent.

Not available space.

Density Is Behaviour

Density should emerge from the user's current activity.

Poor.

Desktop

↓

Dense

Phone

↓

Sparse

Preferred.

Watching

↓

Sparse

Exploring

↓

Dense

The behavioural state determines density.

Presentation simply communicates it.

Sparse Composition

Sparse compositions intentionally communicate very little.

Their purpose is clarity.

Characteristics include:

- one obvious Hero
- minimal competing information
- generous breathing space
- strong hierarchy
- low cognitive effort

Examples include:

- playback
- reading
- listening
- continuing

Sparse compositions should help users disappear into entertainment.

Moderate Composition

Moderate density balances:

- continuation
- exploration
- understanding

Examples include:

- series overview
- current season
- artist profile
- author profile

Most everyday interactions should naturally occupy this level.

Rich Composition

Rich compositions support investigation.

Examples include:

- franchise exploration
- collections
- relationship browsing
- administration
- metadata editing

Rich compositions intentionally communicate more understanding.

However...

Hierarchy should remain obvious.

Rich does **not** mean cluttered.

Density Is Relative

The same information may contribute to different density levels depending upon Context.

Example.

Reviews

Watching.

Low relevance.

Exploring.

High relevance.

The information remains unchanged.

Density changes.

Density Is Adaptive

Density should naturally increase and decrease over time.

Example.

Playback Begins

↓

Sparse

↓

Playback Ends

↓

Moderate

↓

Exploration Begins

↓

Rich

The user should never consciously notice this transition.

They should simply feel that the platform continues supporting their current intent.

Density And Space

Available space influences expression.

It does **not** determine density.

Example.

Television.

Large display.

Current activity.

watching

The Composition should remain sparse.

Additional space should strengthen clarity.

Not introduce unrelated information.

Likewise.

Phone.

Limited display.

Current activity.

Exploring

The Composition should remain rich.

Information should compress.

Not disappear arbitrarily.

Meaning remains stable.

Expression adapts.

Density And Priority

Density should always respect Priority.

Critical

↓

High

↓

Medium

↓

Low

Increasing density should reveal progressively lower-priority information.

Reducing density should hide lower-priority understanding first.

Priority therefore governs density.

Never the reverse.

Density And Breathing Space

Sparse compositions naturally contain more breathing space.

Rich compositions naturally contain less.

However...

Breathing space should never disappear completely.

Every Composition requires visual rhythm.

Without rhythm...

Understanding becomes difficult regardless of information quality.

Density Across Domains

Every entertainment domain should support the same density model.

Television.

Playback

↓

Sparse

Books.

Reading

↓

Sparse

Anime.

Franchise Exploration

↓

Rich

Music.

Artist Discovery

↓

Rich

Different media.

Identical behavioural expectations.

Good Examples

Playback

Hero

↓

Playback

↓

Progress

↓

Timeline

Nothing more.

The Composition remains intentionally quiet.

Series Overview

Hero

↓

Continue

↓

Timeline

↓

Relationships

↓

Cast

↓

Reviews

The Composition communicates significantly more understanding.

Yet hierarchy remains clear.

Administration

Navigation

↓

Current Task

↓

Configuration

↓

Diagnostics

Rich information.

Strong organisation.

No unnecessary decoration.

Anti-patterns

Artificial Density

Adding information simply because additional space exists.

Understanding decreases.

Responsive Density

Changing density purely because screen size changed.

Behaviour becomes inconsistent.

Maximum Density

Displaying every available concept simultaneously.

The Composition becomes a catalogue.

Not an experience.

Decorative Density

Using visual effects to imply richness without increasing understanding.

Complexity increases.

Meaning does not.

Density Model

[mermaid]
flowchart TD

```
Intent
Intent --> Priority
Priority --> Density
Density --> Composition
Composition --> Expressions
Expressions --> Presentation
```

Density emerges from intent.

Presentation communicates density.

Not the other way around.

Relationship To Future Specifications

Future specifications should treat Density as a conceptual property.

Examples include:

- Composition Engine
- Tile Framework
- Material System
- Motion System
- Responsive Behaviour

Density should influence every one of these systems.

None of them should redefine it.

Summary

Density describes how much understanding should be communicated.

Not how tightly information is arranged.

Sparse compositions optimise immersion.

Rich compositions optimise exploration.

The correct density is always determined by the user's current World.

Never by the available pixels.

Review Status

Status

Draft

Next File

08-breathing-space.md

Breathing Space

Purpose

Breathing Space is one of the defining characteristics of the Mosaic Composition Model.

Unlike empty space, which is simply unused interface, Breathing Space is intentional.

It communicates:

- confidence
- hierarchy
- rhythm
- clarity

Breathing Space is therefore considered a compositional concept rather than a visual style.

Its responsibility is to improve understanding.

Not aesthetics.

Definition

Within MDL, **Breathing Space** is defined as:

The intentional preservation of conceptual and visual space that allows understanding to emerge naturally.

Breathing Space exists between concepts.

Not merely between pixels.

It allows the user's attention to move comfortably through a Composition.

Why Breathing Space Exists

Human attention benefits from separation.

Without separation:

- concepts merge
- hierarchy weakens
- rhythm disappears
- understanding slows

Traditional interfaces often attempt to maximise information density.

Mosaic intentionally optimises for comprehension.

Sometimes the fastest way to communicate something is to communicate **less**.

Breathing Space Is Not Empty

One of the most common misconceptions is:

Whitespace

=

Unused Space

Within Mosaic this is incorrect.

Instead:

Breathing Space

=

Reserved Understanding

Breathing Space creates room for:

- hierarchy
- adaptation
- movement
- future evolution

It should therefore be viewed as an active participant within the Composition.

Breathing Space Is Behavioural

Breathing Space should respond to behaviour.

Examples.

Watching

↓

More Space

Exploring

↓

Less Space

The platform should never apply identical spacing simply because identical layouts exist.

Behaviour determines Breathing Space.

Presentation communicates it.

Breathing Space Creates Rhythm

Good compositions possess rhythm.

Example.

Hero

↓

Space

↓

Supporting Information

↓

Space

↓

Context

↓

Space

↓

Peripheral Information

The user naturally understands where one conceptual group ends and another begins.

Rhythm reduces cognitive effort.

Separation Of Meaning

Breathing Space should separate ideas.

Not merely objects.

Good.

Playback

↓

Space

↓

Relationships

Poor.

Playback

Character

Runtime

Reviews

Timeline

Progress

Without conceptual separation...

Everything competes.

Nothing communicates.

Space Around The Hero

The Hero should normally possess the greatest amount of Breathing Space.

This reinforces:

- hierarchy
- confidence
- attention

The Hero should feel intentionally placed.

Not squeezed into remaining interface.

Space surrounding the Hero is therefore part of the Hero.

Not independent from it.

Space And Density

Breathing Space naturally changes with Density.

Sparse Composition.

Large conceptual separation.

Rich Composition.

Reduced conceptual separation.

Importantly...

Reduced does not mean removed.

Every Composition should continue communicating rhythm regardless of density.

Space Supports Adaptation

Adaptive Compositions require room to evolve.

When Information gains Priority:

The Composition should already possess sufficient Breathing Space to accommodate change naturally.

Interfaces filled to capacity have no room to evolve.

They must rebuild.

Mosaic intentionally avoids this behaviour.

Space Supports Movement

Movement requires somewhere to move.

The Composition should therefore preserve enough conceptual space that:

- information may arrive
- information may depart
- hierarchy may evolve

without appearing crowded.

Breathing Space and Movement are therefore closely related.

Future Motion specifications should reinforce this relationship.

Breathing Space Across Devices

Breathing Space should preserve conceptual rhythm rather than physical measurements.

Desktop.

Large physical spacing.

Phone.

Smaller physical spacing.

Equivalent conceptual rhythm.

Television.

Large physical spacing.

Greater viewing distance.

The measurements change.

The understanding remains consistent.

Good Examples

Playback

Hero

↓

Progress

↓

Timeline

Each concept clearly separated.

Nothing competes unnecessarily.

Reading

Current Chapter

↓

Bookmarks

↓

Author

The Composition breathes.

Users naturally understand where attention should move next.

Administration

Navigation

↓

Configuration

↓

Diagnostics

Rich information.

Clear conceptual separation.

The interface remains calm despite increased density.

Anti-patterns

Filling Every Pixel

Every available area contains information.

The interface feels anxious.

Decorative Space

Large gaps without conceptual purpose.

Space should always strengthen understanding.

Uniform Space

Every concept separated equally.

Hierarchy disappears.

No Evolution Space

The Composition cannot adapt because every region is already occupied.

Adaptive behaviour becomes impossible.

Behavioural Model

```
[mermaid]
flowchart TD
```

```
Priority
Priority --> Hierarchy
Hierarchy --> BreathingSpace["Breathing Space"]
BreathingSpace["Breathing Space"] --> Composition
Composition --> Presentation
```

Breathing Space reinforces hierarchy.

It never creates hierarchy.

Relationship To Future Specifications

Future specifications should treat Breathing Space as:

- conceptual
- adaptive
- behavioural

Examples include:

- Material System
- Composition Engine
- Tile Framework
- Responsive Behaviour
- Motion System

Every future implementation should preserve conceptual rhythm before optimising physical layout.

Summary

Breathing Space is not unused interface.

It is intentional understanding.

It creates:

- rhythm
- clarity
- confidence

- adaptability

The absence of unnecessary information is considered an active design decision.

Not a missed opportunity.

Good Compositions communicate just enough.

Breathing Space ensures users have room to understand what matters.

Review Status

Status

Draft

Next File

09-composition-solving.md

Composition Solving

Purpose

Previous chapters defined what a Composition is and how it should behave.

This chapter introduces one of the most important concepts within the Mosaic Design Language.

The **Composition Solver**.

Unlike traditional layout engines, the Composition Solver does not solve geometry.

It solves understanding.

Its responsibility is to determine the best possible Composition for the user's current World before any visual layout occurs.

This chapter represents the conceptual foundation for the future **MDS Composition Engine**.

Definition

Within MDL, **Composition Solving** is defined as:

The process of determining the optimal organisation of understanding for the user's current World.

The Composition Solver answers one question.

Given everything we currently know, what should the user understand first?

Why A Solver Exists

Traditional interfaces frequently rely upon manually authored layouts.

Examples.

Home

↓

Fixed Sections

↓

Render

or

Screen

↓

Grid

↓

Populate Widgets

These systems assume every user requires the same organisation.

Mosaic intentionally rejects this assumption.

Every World is different.

Every Focus is different.

Every Context is different.

The Composition should therefore be solved.

Not hardcoded.

Solving Understanding

The Composition Solver is **not** a layout algorithm.

It never asks:

- Which row?
- Which column?
- Which pixel?

Instead it asks:

- What matters?
- Why?
- What supports it?
- What can wait?
- What should disappear?

The output is understanding.

Presentation is solved afterwards.

Inputs

The Composition Solver receives conceptual inputs.

[text]
world

↓

Focus

↓

Context

↓

Information

↓

Relationships

↓

Priority

↓

Available Capabilities

Notice that layout information is intentionally absent.

The solver operates entirely within the conceptual domain.

Outputs

The Composition Solver produces:

- Hero
- Hierarchy
- Priority Ordering
- Groupings
- Adaptive Density
- Expressions
- Behavioural Intent

These outputs are consumed later by the Design System.

The solver never creates interface directly.

Solving Order

Every Composition should be solved using the same conceptual sequence.

[text]

1.

Current World

↓

2.

Current Focus

↓

3.

Current Context

↓

4.

Relevant Information

↓

5.

Relationships

↓

6.

Priority

↓

7.

Composition

↓

8.

Expressions

↓

9.

Presentation

Skipping stages weakens understanding.

The Solver Never Guesses

The Composition Solver should never invent understanding.

It should only organise understanding that already exists.

Poor.

Maybe this is important.

Preferred.

Current Context indicates this is important.

Understanding should emerge from evidence.

Not assumption.

Solving Is Deterministic

Given identical inputs...

The Composition Solver should always produce the same conceptual output.

Example.

world

↓

Frieren

↓

Watching

↓

Episode 15

Every client should produce the same Composition.

Presentation may differ.

Understanding must remain identical.

This deterministic behaviour is fundamental to preserving the Mosaic Mental Model across platforms.

Solving Priority

Priority should be determined before solving Composition.

Example.

Episode

High

↓

Timeline

Medium

↓

Reviews

Low

The solver should never determine Priority.

Priority is an input.

Composition is the result.

Solving Relationships

Relationships significantly influence Composition.

Example.

Episode

↓

Manga

↓

Author

↓

Next Episode

These relationships naturally suggest:

Continue

↓

Timeline

↓

Related Works

The solver should therefore prefer meaningful relationships over arbitrary ordering.

Solving For Intent

The Composition Solver should optimise the user's current intent.

Example.

Intent.

Continue Watching

Preferred Composition.

Playback

↓

Progress

↓

Next Episode

Not.

Trending

↓

Collections

↓

Statistics

Intent always possesses higher authority than inventory.

Solving Across Devices

The Composition Solver should remain device independent.

Desktop.

Composition

↓

Desktop Presentation

Phone.

Composition

↓

Phone Presentation

Television.

Composition



TV Presentation

The solved understanding remains identical.

Only the presentation changes.

Solving With Plugins

Plugins should never influence the solving process directly.

Plugins contribute:

- Information
- Relationships

The platform determines:

- Priority
- Hierarchy
- Hero
- Expressions

This preserves one coherent behavioural language.

Extensions strengthen the World.

They do not redesign it.

Failure Conditions

A Composition should be considered unsolved when:

- multiple Heroes compete
- hierarchy is unclear
- unrelated information dominates
- understanding depends upon presentation
- users must consciously search for what matters

The solver has failed if users ask:

"Where should I look?"

Good Examples

Watching

Inputs.

Current Episode

↓

Progress

↓

Next Episode

↓

Cast

↓

Reviews

Solved Composition.

Hero

↓

Progress

↓

Timeline

↓

Relationships

↓

Metadata

Understanding emerges naturally.

Reading

Inputs.

Current Chapter

↓

Bookmarks

↓

Series

↓

Author

↓

Statistics

Solved Composition.

Hero

↓

Reading Progress

↓

Bookmarks

↓

Series

↓

Statistics

Again...

Understanding leads.

Anti-patterns

Layout Solving

Choosing positions before determining importance.

Component Solving

Thinking in:

- cards
- shelves
- widgets

instead of understanding.

Responsive Solving

Changing hierarchy because screen size changed.

Meaning should remain constant.

Only expression adapts.

Plugin Solving

Extensions determining:

- Hero
- Priority

- Hierarchy

The platform loses ownership of understanding.

Conceptual Solver

[mermaid]
flowchart TD

```
World
World --> Focus
Focus --> Context
Context --> Information
Information --> Relationships
Relationships --> Priority
Priority --> CompositionSolver["Composition Solver"]
CompositionSolver["Composition Solver"] --> Expressions
Expressions --> Presentation
```

The Composition Solver exists entirely before presentation.

Its output is understanding.

Not interface.

Relationship To MDS

This chapter intentionally avoids implementation.

The future **MDS Composition Engine** will define:

- algorithms
- heuristics
- runtime solving
- optimisation
- caching
- adaptive rendering

MDL defines only the conceptual expectations.

MDS defines the implementation.

Summary

Composition Solving is one of the defining architectural ideas of Mosaic.

Instead of manually arranging interface...

Mosaic first solves understanding.

Everything else becomes a consequence of that decision.

Users should therefore experience interfaces that feel naturally organised around their current World rather than manually designed around arbitrary layouts.

Review Status

Status

Draft

Next File

10-device-independence.md

Device Independence

Purpose

One of the primary goals of the Mosaic Design Language is to ensure that the user's entertainment World remains recognisable regardless of the device through which it is experienced.

This chapter formalises one of the core architectural promises of Mosaic.

The World never changes.

Only its expression changes.

Everything else within the Design Language exists to uphold this promise.

Definition

Within MDL, **Device Independence** is defined as:

The ability for the same World, Composition and behavioural model to be expressed consistently across different interaction environments without redefining the user's understanding.

Device Independence is not:

- responsive design
- adaptive layout
- mobile-first design

Those are implementation strategies.

Device Independence is a conceptual guarantee.

Why Device Independence Exists

People no longer consume entertainment on a single device.

A single experience may begin on:

- a television

continue on:

- a tablet

resume on:

- a phone

and conclude on:

- a desktop

Users should never feel they have entered four different products.

They should simply feel that:

Their World followed them.

The World Is Constant

Every client should communicate the same underlying World.

World

↓

Focus

↓

Context

↓

Information

↓

Relationships

These concepts never change because the device changed.

Only their presentation changes.

Expression Changes

The following layers may adapt according to device capabilities.

Expressions

↓

Presentation

↓

Components

↓

Layout

↓

Materials

The following layers should remain stable.

World

Focus

Context

Relationships

Priority

Composition

This separation preserves conceptual continuity.

Device Classes

Future implementations may define device classes.

Examples include:

Television

Characteristics:

- distance
- remote control
- large artwork
- immersive viewing

Desktop

Characteristics:

- precision input
- large workspace
- multitasking

Tablet

Characteristics:

- touch
- portability
- reading

Mobile

Characteristics:

- one-handed interaction
- constrained space
- interruption

Voice

Characteristics:

- no visual interface

- conversational output
- sequential interaction

Despite these differences...

The user's World remains identical.

Behaviour Before Device

A common implementation mistake is:

Phone

↓

Design Experience

Instead:

User World

↓

Composition

↓

Expression

↓

Phone

The behavioural model should never be determined by hardware.

Hardware determines only how behaviour is communicated.

The Hero

Every client should communicate the same Hero.

Examples.

Television.

Large Poster

Desktop.

Expanded Hero

Phone.

Compact Hero

Voice.

"Continue watching Frieren?"

The Hero remains conceptually identical.

Only its expression changes.

Navigation

Navigation should preserve behavioural identity.

Desktop.

Sidebar

Mobile.

Bottom Navigation

Television.

Overlay

The Navigation Anchor remains behaviourally identical.

Only presentation differs.

Density

Different devices should communicate the same density differently.

Example.

Rich Composition.

Desktop.

Many Expressions

Phone.

Fewer simultaneous Expressions

↓

Progressive Disclosure

The Composition remains rich.

The expression becomes more compact.

Accessibility

Accessibility should also preserve Device Independence.

Examples include:

- reduced motion
- larger text
- screen readers
- high contrast

These adaptations should change presentation.

They should never redefine:

- hierarchy
- priority
- behaviour

Users should experience the same understanding regardless of accessibility preferences.

Cross-Device Continuity

A user beginning an experience on one device should naturally continue on another.

Example.

Television

↓

Playback

↓

Phone

↓

Continue Watching

The platform should restore:

- Focus
- Context
- Progress
- Relationships

The user should never reconstruct their World simply because the device changed.

Plugins

Extensions should remain entirely device independent.

Plugins contribute:

- Information
- Relationships

The platform determines:

- Expressions
- Presentation
- Components
- Layout

Plugins should therefore require no device-specific interface logic.

This significantly reduces ecosystem complexity.

Anti-patterns

Separate Products

Desktop and mobile behaving like unrelated applications.

Device-Driven Behaviour

Interaction changes because screen size changed.

Device-Specific Concepts

Introducing concepts that exist only on one platform.

The Mental Model fragments.

Plugin UI

Extensions rendering completely different experiences on different devices.

The platform loses ownership of consistency.

Behavioural Model

[mermaid]
flowchart TD

```
World
World --> Composition
Composition --> Expression
Expression --> Device
Device --> Presentation
Device --> Desktop
Device --> Mobile
Device --> Television
Device --> Voice
```

Notice that the device appears only after Expression selection.

Understanding always precedes hardware.

Relationship To Future Specifications

Future MDS specifications should preserve this separation.

Examples include:

- Composition Engine
- Motion System

- Material System
- Component Library
- Runtime Adaptation

Every implementation should ask:

How should this device communicate the Composition?

Never:

How should this device redefine the Composition?

Summary

Device Independence is one of the defining architectural promises of Mosaic.

Users should recognise:

- their World
- their Focus
- their Context
- their Composition

immediately...

regardless of whether they are using:

- a television
- a desktop
- a tablet
- a phone
- a future interaction device

The platform adapts.

The World remains constant.

Review Status

Status

Draft

Next File

11-governance.md

Composition Model Governance

Purpose

The Composition Model defines how understanding is organised.

Unlike presentation, which is expected to evolve regularly, composition represents long-lived product behaviour.

Changing composition changes how people think.

Consequently, changes to the Composition Model should be considered architectural decisions rather than visual refinements.

This chapter defines how Composition should evolve throughout the lifetime of Mosaic.

Composition Is Product Architecture

Within Mosaic, Composition is treated as product architecture.

Presentation implements Composition.

It does not replace it.

Examples.

Presentation may change:

- Acrylic becomes another material.
- Typography evolves.
- Components are redesigned.
- New devices appear.

Composition should remain recognisable.

Users should continue understanding:

- what matters
- what changed
- where to continue

without relearning the platform.

Stability

Expected lifespan.

Artefact	Expected Lifetime
Components	Months
Materials	Months

Artefact	Expected Lifetime
Layouts	Months
Presentation	Months
Composition	Years
Mental Model	Decades
Vision	Decades

Composition should therefore evolve deliberately rather than reactively.

What Requires Review

The following changes require formal Composition review.

- New hierarchy models
- New priority systems
- Hero behaviour
- Anchor behaviour
- Adaptive composition rules
- Density model
- Composition solver behaviour
- Cross-device composition rules

These changes affect every Mosaic client.

What Does Not Require Review

The following normally belong to MDS.

- Component redesign
- Material updates
- Motion tuning
- Typography
- Colours
- Spacing
- Responsive implementation
- Rendering optimisation

These concern expression.

Not composition.

Composition Review Questions

Every proposed compositional change should answer the following questions.

Question One

Does this strengthen understanding?

Question Two

Does this reduce cognitive effort?

Question Three

Does this preserve the user's World?

Question Four

Does this reinforce hierarchy?

Question Five

Would another contributor naturally solve this problem the same way after reading MDL?

If the answer is no...

The Composition Model probably requires refinement before implementation.

Composition Drift

Composition drift occurs when multiple parts of the platform begin organising understanding differently.

Examples include:

- multiple hero strategies
- competing hierarchy systems
- different priority rules
- inconsistent grouping
- device-specific compositions

Composition drift is particularly dangerous because users rarely notice it consciously.

Instead they simply feel that the product has become harder to use.

Composition Debt

Composition debt accumulates when:

- exceptions multiply
- hierarchy becomes inconsistent
- priority rules diverge
- unrelated concepts compete
- adaptive behaviour becomes unpredictable

Unlike technical debt...

Composition debt affects every interaction.

It should therefore be treated as a first-class architectural concern.

Evolution

The Composition Model should evolve through refinement.

Preferred.

Existing Hierarchy

↓

Improved Hierarchy

Avoid.

Existing Hierarchy

↓

Alternative Hierarchy

↓

Special Case

↓

Exception

The platform should possess one compositional language.

Not several.

Extensions

Extensions should strengthen Composition.

They should never redefine it.

Plugins contribute:

- Information
- Relationships

The Composition Solver determines:

- Hero

- Priority
- Hierarchy
- Grouping
- Expressions

This separation ensures that every extension naturally feels like part of the same World.

Governance Workflow

[mermaid]
flowchart TD

```

Proposal
Proposal --> CompositionReview["Composition Review"]
CompositionReview["Composition Review"] --> ExistingModel["Existing Model?"]
ExistingModel["Existing Model?"] -->|Yes| Refine
ExistingModel["Existing Model?"] -->|No| ADR
ADR --> SpecificationUpdate["Specification Update"]
SpecificationUpdate["Specification Update"] --> MDSReview["MDS Review"]

```

The preferred outcome is refinement.

Introducing new compositional concepts should remain rare.

Success Criteria

The Composition Model succeeds when:

- every client communicates the same hierarchy
- plugins naturally integrate
- contributors rarely invent new composition concepts
- users instinctively know where to look
- adaptive behaviour feels inevitable rather than surprising

Composition should eventually become invisible.

Users should experience understanding rather than organisation.

Architectural Decisions

ADR	Decision
ADR-067	Composition is considered long-lived product architecture.
ADR-068	Hierarchy and priority are platform responsibilities rather than presentation concerns.
ADR-069	Composition should evolve through refinement rather than replacement.
ADR-070	Extensions contribute understanding rather than compositional behaviour.

Review Status

Status

Draft

Next File

12-adrs.md

Architectural Decision Records

Purpose

The Architectural Decision Records (ADRs) contained within MDL-005 preserve the reasoning behind the Composition Model.

Where previous specifications established:

- Vision
- Principles
- Mental Model
- Interaction

MDL-005 establishes **how understanding is organised**.

These ADRs document why the Composition Model was designed the way it was, ensuring future contributors understand the architectural intent before proposing changes.

Composition is considered one of the defining architectural systems of Mosaic.

Its decisions should therefore remain traceable throughout the lifetime of the project.

ADR Format

Every ADR follows the standard Mosaic structure.

[text]

ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications

Each ADR records exactly one significant compositional decision.

ADR-071

Title

Composition Organises Understanding Rather Than Interface

Status

Accepted

Context

Traditional interface frameworks typically organise interface components directly.

This tightly couples understanding with implementation.

Decision

Composition becomes a conceptual layer that organises understanding before presentation exists.

Consequences

Presentation technologies may evolve independently while preserving identical conceptual organisation.

ADR-072

Title

Hierarchy Emerges From Priority

Status

Accepted

Context

Many design systems assign hierarchy manually.

This frequently produces inconsistent experiences.

Decision

Hierarchy becomes a direct consequence of Priority.

Priority itself emerges from:

- Focus
- Context
- Relationships

Consequences

Hierarchy naturally adapts as the user's World evolves.

No manual hierarchy management should normally be required.

ADR-073

Title

Hero Is An Emergent Concept

Status

Accepted

Context

Traditional interfaces frequently hardcode hero regions.

This weakens adaptability.

Decision

The Hero becomes the highest-priority Expression within the current Composition.

Consequences

Heroes evolve naturally.

They are never manually assigned.

Different clients remain conceptually identical despite differing visual implementations.

ADR-074

Title

Anchors Preserve Orientation

Status

Accepted

Context

Highly adaptive interfaces frequently become disorientating.

Founder workshops repeatedly identified the need for behavioural stability.

Decision

Stable Anchors become first-class compositional concepts.

Anchors preserve orientation while surrounding information adapts.

Consequences

Adaptive compositions remain understandable even as they evolve continuously.

ADR-075

Title

Density Is Behavioural Rather Than Visual

Status

Accepted

Context

Density is traditionally treated as spacing or compact mode.

This tightly couples understanding with presentation.

Decision

Density becomes a conceptual property describing how much understanding should be communicated.

Presentation determines how density is expressed.

Consequences

Rich and sparse compositions remain consistent across every supported platform.

ADR-076

Title

Composition Is Solved Rather Than Authored

Status

Accepted

Context

Static layouts cannot effectively represent continuously changing Worlds.

Founder workshops consistently favoured adaptive understanding over static screens.

Decision

Composition becomes the output of a conceptual solving process.

The Composition Solver determines:

- hierarchy
- priority
- grouping
- Hero
- Expressions

Consequences

Future runtime systems remain adaptive while preserving one coherent compositional language.

ADR-077

Title

Plugins Contribute Knowledge Rather Than Layout

Status

Accepted

Context

Allowing extensions to generate interface fragments inevitably fragments product identity.

Decision

Plugins contribute:

- Information
- Relationships

The platform remains solely responsible for:

- Composition
- Priority
- Hierarchy
- Expressions

Consequences

Every extension inherits the same compositional language.

The ecosystem scales without creating competing interface paradigms.

ADR-078

Title

Composition Is Device Independent

Status

Accepted

Context

Users increasingly consume entertainment across multiple devices.

Maintaining separate composition models would fragment understanding.

Decision

Composition becomes device independent.

Only Presentation adapts.

Consequences

Users experience one consistent World regardless of device.

ADR-079

Title

Breathing Space Is A Compositional Concept

Status

Accepted

Context

Whitespace is traditionally considered a visual concern.

Founder workshops consistently described empty space as supporting understanding rather than aesthetics.

Decision

Breathing Space becomes part of Composition rather than visual styling.

Consequences

Future Material Systems and Layout Engines should preserve conceptual rhythm before optimising visual density.

ADR Relationships

[mermaid]
flowchart TD

ADR071["Understanding"]

ADR072["Priority"]

ADR073["Hero"]

ADR074["Anchors"]

ADR075["Density"]

ADR076["Solver"]

ADR077["Plugins"]

ADR078["Device Independence"]

ADR079["Breathing Space"]

ADR071 --> ADR072

ADR072 --> ADR073

ADR072 --> ADR075

ADR073 --> ADR074

ADR075 --> ADR079

ADR074 --> ADR076

ADR076 --> ADR077

ADR076 --> ADR078

Every compositional decision reinforces the same objective.

Understanding before presentation.

Future ADRs

Future Composition ADRs are expected to formalise:

- Composition Scoring
- Relationship Weighting
- Expression Selection
- Runtime Adaptation
- Multi-User Composition
- Shared Composition
- Predictive Composition
- Accessibility Composition

These intentionally remain outside MDL-005 Version 0.1.

ADR Governance

Composition ADRs should change only when:

- repeated user research demonstrates misunderstanding,
- behavioural evolution requires conceptual refinement,
- the Mental Model changes,
- multiple specifications repeatedly conflict.

Presentation concerns should never trigger changes to the Composition Model.

Summary

The ADRs within MDL-005 define one of the most distinctive architectural characteristics of Mosaic.

Rather than arranging interface...

Mosaic arranges understanding.

Every future implementation should preserve this distinction.

Doing so allows the platform to evolve visually while remaining conceptually stable for years to come.

Review Status

Status

Draft

Next File

13-contributor-guidance.md

Contributor Guidance

Purpose

The Composition Model is one of the most frequently applied specifications within the Mosaic Design Language.

Unlike the Vision or Principles, contributors will make compositional decisions almost every day.

This chapter exists to ensure those decisions remain consistent across:

- core development
- community extensions
- future client applications
- future engineering teams

The goal is not to make every interface identical.

The goal is to ensure every interface **feels** unmistakably Mosaic.

Design Understanding First

Never begin by asking:

Where should this component go?

Instead ask:

What understanding should this composition create?

Composition exists to communicate understanding.

Everything else is implementation.

Think In Understanding

When designing a new experience, contributors should follow the same conceptual sequence.

User Intent

↓

World

↓

Focus

↓

Context

↓

Information

↓

Relationships

↓

Priority

↓

Composition

↓

Expressions

↓

Presentation

If implementation begins before this sequence has been completed, the resulting experience will usually become implementation-driven rather than user-driven.

Never Design Pages

Avoid thinking:

Home Page

↓

Details Page

↓

Settings Page

Instead think:

Current World

↓

Current Focus

↓

Current Understanding

The interface should emerge naturally from those concepts.

If a contributor begins drawing screens before understanding the Composition, they are working in the wrong order.

Design Hierarchy Before Layout

Contributors should determine:

- what matters
- why it matters
- what supports it

- what can wait

before considering:

- grids
- rows
- columns
- spacing
- breakpoints

Hierarchy always precedes geometry.

Every Composition Tells One Story

A Composition should communicate one primary idea.

Examples.

Continue Watching

Continue Reading

Discover This Franchise

Manage Your Library

If several unrelated stories compete for attention, the Composition should be reconsidered.

Preserve The Hero

Every Composition should naturally communicate one Hero.

Before introducing new information ask:

Does this strengthen the Hero...

or...

Does this compete with it?

Supporting information should reinforce the Hero.

Not challenge it.

Respect Anchors

Anchors preserve orientation.

They should remain behaviourally stable while surrounding information evolves.

Contributors should avoid introducing unnecessary movement or reorganisation around:

- Navigation
- Search

- Current Focus
- Playback

The more adaptive the Composition becomes...

The more valuable Anchors become.

Think In Priority

Contributors should never manually decide that something is visually important.

Instead determine:

Priority

↓

Hierarchy

↓

Composition

↓

Presentation

Visual emphasis should emerge naturally from conceptual importance.

Think In Expressions

Do not think:

Timeline Tile

Think:

Timeline Expression

The Expression may later become:

- Tile
- Shelf
- Hero Detail
- Voice Output
- Notification

Expressions survive implementation.

Components rarely do.

Plugins

Extension authors should contribute:

- Information
- Relationships

They should never attempt to contribute:

- layout
- hierarchy
- hero
- anchors
- density

Those responsibilities belong exclusively to the Composition Model.

Plugins enrich understanding.

They do not organise it.

Device Independence

Before implementing a Composition ask:

Would this still make sense on a television?

On a phone?

On a voice interface?

If the answer is no...

The Composition probably depends upon presentation rather than understanding.

Common Design Smells

The following usually indicate the Composition Model is being bypassed.

Multiple Heroes

Several concepts compete for attention.

Equal Hierarchy

Every region appears equally important.

Layout Before Meaning

Grid created.

Meaning added afterwards.

Plugin Layout

Extension defines interface.

Device Thinking

Composition changes because screen size changed.

Meaning should remain constant.

Only presentation adapts.

Composition Review Questions

Before implementation ask:

- What is the Hero?
- Why is it the Hero?
- What supports it?
- What belongs in the background?
- Which information could disappear without reducing understanding?
- Which information must never disappear?
- Does this Composition communicate one story or several?

If these questions cannot be answered confidently, continue refining the Composition before implementing it.

Composition Checklist

Every significant Composition should satisfy the following.

- ☐ One Hero exists.
- ☐ Priority is obvious.
- ☐ Hierarchy follows Priority.
- ☐ Anchors preserve orientation.
- ☐ Density matches user intent.
- ☐ Breathing Space reinforces understanding.
- ☐ Expressions remain reusable.
- ☐ Presentation remains replaceable.
- ☐ Plugins contribute knowledge rather than layout.
- ☐ Device changes do not alter understanding.

Final Guidance

The Composition Model should become so deeply understood that contributors stop consciously applying it.

Instead of thinking:

"Where should this go?"

they should naturally think:

"What understanding am I trying to create?"

When every contributor begins asking that question instinctively, Mosaic will cease feeling like a collection of interfaces.

It will begin feeling like one coherent world.

That is the ultimate objective of the Composition Model.

Review Status

Status

Draft

Next File

glossary.md

Glossary

Purpose

This glossary defines the compositional terminology introduced by **MDL-005 — Composition Model**.

Unlike previous glossaries, this document focuses specifically on how understanding is organised before it becomes presentation.

The terms defined here should be considered normative throughout all future Composition Engine, Tile Framework and Design System specifications.

A

Adaptive Composition

A Composition capable of reorganising itself in response to changes in:

- Focus
- Context
- Priority
- Available Space
- Device
- User Intent

while preserving understanding.

Adaptive Composition changes organisation.

It does not redefine meaning.

Anchor

A stable conceptual region that preserves orientation while surrounding Compositions evolve.

Anchors communicate stability rather than attention.

Examples include:

- Navigation
- Search
- Current Focus
- Playback

B

Breathing Space

The intentional preservation of conceptual space that improves understanding.

Breathing Space exists between ideas rather than merely between interface elements.

It reinforces:

- hierarchy
- rhythm
- clarity
- adaptability

C

Composition

The intentional organisation of understanding into a coherent experience.

Composition exists before:

- layout
- components
- materials
- presentation

It communicates meaning rather than geometry.

Composition Evolution

The continuous refinement of a Composition as the user's World evolves.

Composition Evolution preserves continuity while allowing understanding to change naturally.

Composition Solver

The conceptual process responsible for determining the optimal organisation of understanding.

The Composition Solver produces:

- Hero
- Hierarchy
- Priority
- Groupings
- Expressions

It does not produce interface.

D

Density

The amount of conceptual understanding intentionally communicated within a Composition.

Density is behavioural.

Not visual.

E

Expression

The conceptual communication pattern selected by the Composition.

Expressions remain independent from:

- components
- devices
- layouts

Examples include:

- Timeline
- Progress
- Metadata
- Notification

G

Grouping

The organisation of related Information into coherent conceptual units.

Grouping exists to reduce interpretation.

Not simply organise interface.

H

Hero

The highest-priority Expression within the current Composition.

The Hero represents:

What deserves attention now.

A Hero is never manually assigned.

It naturally emerges from Priority.

Hierarchy

The intentional ordering of attention according to the user's current World.

Hierarchy communicates Priority.

It does not create it.

P

Peripheral Information

Information currently possessing the lowest conceptual priority.

Peripheral Information remains available without competing for attention.

Priority

The relative importance of Information within the user's current World.

Priority changes as:

- Focus changes
- Context changes
- Time passes
- Relationships evolve

Priority exists before Hierarchy.

Progressive Disclosure

A compositional strategy in which Information becomes visible as it becomes useful.

Progressive Disclosure reduces cognitive effort while preserving access to richer understanding.

R

Responsive Presentation

A presentation strategy adapting Expressions to different devices.

Responsive Presentation should never redefine Composition.

S

Sparse Composition

A Composition intentionally communicating only essential understanding.

Sparse Compositions prioritise:

- immersion
- clarity
- continuation

Supporting Information

Information directly reinforcing the current Hero.

Supporting Information exists because it helps users continue their current journey.

T

Temporal Priority

Priority influenced by the passage of time.

Examples include:

- upcoming releases
- countdowns
- newly available episodes

Temporal Priority allows Compositions to evolve naturally without requiring user interaction.

U

Understanding

The successful communication of the user's current World.

Every Composition exists to maximise understanding while minimising cognitive effort.

Understanding is considered the primary success metric of the Composition Model.

Cross References

Specification	Primary Concepts
MDL-001 Vision	Companion, Immersion
MDL-002 Principles	Context, Content, Behaviour
MDL-003 Mental Model	World, Focus, Information
MDL-004 Interaction Model	Behaviour, Continuity, Evolution
MDS Composition Engine	Runtime Composition
MDS Tile Framework	Expressions, Presentation

Terminology Rules

Contributors should:

- describe understanding before interface
- describe hierarchy before layout
- describe priority before visual emphasis
- describe expressions before components

Composition terminology should remain independent from implementation.

Review Status

Status

Draft

Next File

references.md

References

Purpose

This document records the architectural influences, design philosophies and conceptual references that informed **MDL-005 — Composition Model**.

Unlike implementation documentation, these references explain *why* the Composition Model exists rather than *how* it should be implemented.

MDL intentionally draws inspiration from multiple disciplines.

The resulting Composition Model is unique to Mosaic.

Reading Order

Contributors should approach references in the following order.

1. MDL Specifications 2. Information Architecture 3. Human Perception 4. Interaction Design 5. Systems Architecture 6. Implementation Specifications

The Mosaic Design Language always remains the authoritative source.

Internal References

MDL-001 — Vision

Provides:

- Product vision
- Immersion
- Companion philosophy
- Long-term product objectives

Every Composition should reinforce the experience described within MDL-001.

MDL-002 — Principles

Provides:

- Context Before Prediction
- Content Leads
- Every Feature Earns Its Place
- Be A Companion

The Composition Model should be viewed as the practical application of these principles.

MDL-003 — Mental Model

Provides:

- World
- Focus
- Context
- Information
- Relationships
- Expressions

MDL-005 organises these concepts.

It does not redefine them.

MDL-004 — Interaction Model

Provides:

- Continuity
- Behaviour
- Focus Transitions
- Context Evolution
- Movement

MDL-005 determines how these behavioural changes become understandable compositions.

Future Specifications

The following specifications directly depend upon MDL-005.

- MDS-001 Design Token Architecture
- MDS-002 Material System
- MDS-003 Composition Engine
- MDS-004 Runtime Atmosphere
- MDS-007 Tile Framework
- MDS-008 Component Library

These specifications should implement the Composition Model rather than reinterpret it.

Information Architecture

The Composition Model draws heavily from the discipline of Information Architecture.

Primary influences include:

- understanding before interface
- hierarchy before layout
- conceptual grouping
- progressive disclosure
- information scent

Mosaic intentionally extends these ideas into a runtime composition system capable of continuously reorganising understanding.

Human Perception

Several principles of human perception influenced the Composition Model.

Examples include:

- visual hierarchy
- progressive attention
- chunking
- spatial memory
- cognitive load reduction

These concepts influenced compositional thinking rather than visual styling.

Editorial Design

Editorial design significantly influenced the Composition Model.

Editors continually decide:

- what deserves attention
- what supports understanding
- what should wait

Composition performs the same role.

Instead of organising articles...

It organises understanding.

This editorial mindset differentiates Mosaic from grid-based dashboard systems.

Systems Thinking

The Composition Model intentionally adopts systems thinking.

Examples include:

- Hero emerging from Priority

- Hierarchy emerging from Relationships
- Density emerging from Context
- Expressions emerging from Composition

Nothing should exist in isolation.

Every compositional decision should emerge naturally from the concepts preceding it.

Design Systems

The organisation of MDL-005 was informed by mature design systems that distinguish:

- conceptual hierarchy
- interaction
- implementation
- presentation

rather than treating layout as the primary architectural concern.

Mosaic intentionally extends this approach by introducing runtime composition.

Software Architecture

Although MDL intentionally avoids implementation details, several architectural concepts influenced the Composition Model.

Examples include:

- Separation of Concerns
- Information-Driven Architecture
- Constraint Solving
- Event-Driven Systems
- Domain-Driven Design
- Documentation as Code

These concepts influenced documentation structure.

They do not prescribe implementation.

Mosaic-Specific Influences

The Composition Model emerged directly from founder workshops.

Significant discoveries included:

- the World as the primary conceptual object
- Hero emerging from Priority

- Information before interface
- Relationships before navigation
- Expressions before presentation
- Composition solving rather than authored layouts
- Plugins contributing understanding rather than interface

These discoveries collectively distinguish Mosaic from conventional media platforms.

Relationship To The Platform

The broader Mosaic platform architecture introduces concepts such as:

- GraphQL
- Extensions
- Runtime composition
- Adaptive presentation
- Information graphs

The Composition Model intentionally remains implementation independent while providing the conceptual framework that those systems should implement.

Normative References

Required reading before contributing to MDL-005.

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model

Informative References

Future contributors may also wish to review:

- MDS-003 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library
- MDS-005 Motion System

These documents describe how the conceptual Composition Model becomes runtime behaviour.

Living Document

This reference list should remain intentionally concise.

References should only be introduced when they materially influence:

- compositional architecture
- hierarchy
- behavioural organisation
- conceptual understanding

The purpose of this document is to preserve architectural context rather than become an exhaustive bibliography.

Completion

This concludes **MDL-005 — Composition Model**.

With the completion of MDL-005, the conceptual layer of the Mosaic Design Language is now established.

The next specification begins the transition from philosophy into implementation:

MDS-001 — Design Token Architecture

From this point onwards the documentation shifts from answering:

- Why?
- How should we think?
- How should the platform behave?

to answering:

- How should these concepts be implemented?
- How should the runtime consume them?
- How should every Mosaic client express them consistently?

The remaining MDS specifications build directly upon the five MDL specifications and collectively form the implementation blueprint for the Mosaic platform.